

Authentication Failures

En esta sesión vamos a aumentar problemas, vulnerabilidades y pruebas que debemos hacer para analizar e identificar vulnerabilidades asociadas a fallos en la autenticación de aplicaciones web.

Para ello, el primer punto que vamos a atacar de esta sesión va a ser el ataque de fuerza bruta.

Brute force attacks

- **Execution of several attempts of authentication with one known user and different passwords**
- **Tools**
 - **BurpSuite -> Intruder**
- **Mitigation**
 - **Blocking a user after 3-5 login attempts**

En este primer tipo de prueba vamos a realizar un ataque que es muy conocido y que consiste al fin y al cabo en la repetición masiva de una petición en la que se intenta autenticar un usuario que conocemos, de forma que habiendo validado previamente o conociendo la existencia previa de este usuario, vamos a intentar adivinar su contraseña con muchos intentos también.

Para realizar este ataque, esta prueba, la herramienta más usada y más conocida es Burpsuite con su funcionalidad Intruder.

Otro aspecto a considerar es la mitigación de esta vulnerabilidad, que principalmente se trata del bloqueo del usuario afectado temporalmente durante unos minutos, de forma que las peticiones que se están replicando intentando autenticar al usuario afectado no sean validadas o sean desechadas por el propio servidor.

Para realizar un ejemplo de esta prueba vamos a hacer uso de la aplicación DVWA como el entorno víctima y vamos a usar la máquina de Kali Linux como máquina atacante.

Práctica:

Aquí tenemos la aplicación DWA corriendo en nuestra máquina Kali, ya que tenemos visibilidad, la tenemos accesible gracias al entorno de Metasploitable que está habilitado y como es un servicio que tiene configurado, vamos a aprovecharlo.

También tenemos la aplicación Burpsuite, cuyo tráfico estamos proxyficando en el navegador con FoxyProxy y lo estamos interceptando, de forma que al intentar autenticar a un usuario admin, los usuarios tales como admin, root, administrator, administrador, son conocidos por ser usuarios por defecto en muchas ocasiones usuario test, usuario guest, son usuarios cuyo uso, si no se va a usar ese usuario es mejor deshabilitarlo o darlo de baja.

En este caso vamos a probar con el usuario admin y la contraseña admin, clásicas credenciales de admin/admin. En este caso nos dice que el login ha fallado, pero nosotros hemos interceptado la petición y vamos a enviarla a la funcionalidad intruder.

Estamos en la tab positions, en este caso como sabemos previamente que el usuario admin existe, lo que vamos a cambiar es el valor del parámetro password, seleccionando el valor parámetro (admin) y dándole a Add a la derecha.

Vamos al tab Payloads, y en los Payload settings añadimos manualmente los passwords que queremos probar. De forma que vamos a realizar intentos con por ejemplo la contraseña root, toor, contraseñas que nos pueden sonar de pruebas que hayamos realizado contraseña por ejemplo admin1, admin2, admin3 y también otra contraseña que nos puede interesar es password.

Para ello hay muchas aplicaciones que a la hora de autenticar los usuarios lo que hacen es una redirección cuando la autenticación es satisfactoria, por ello también hay que tener en cuenta que en las configuraciones de este intruder, en la tab Settings, podemos indicar que podemos seguir la redirección de la petición que

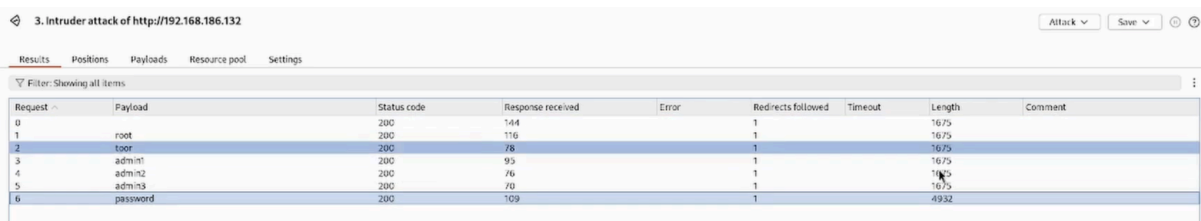
vayamos a realizar en nuestro ataque. Vamos a la sección de Redirection y chequeamos Process cookies in redirections.



Una vez hecho esto vamos a hacer como podemos ver aquí, seis pruebas, seis intentos de autenticación con contraseñas.



Comenzamos este ataque.



Request	Payload	Status code	Response received	Error	Redirects followed	Timeout	Length	Comment
0		200	144		1		1675	
1	root	200	116		1		1675	
2	toor	200	78		1		1675	
3	admin1	200	95		1		1675	
4	admin2	200	76		1		1675	
5	admin3	200	70		1		1675	
6	password	200	109		1		4932	

En este caso podemos observar que la longitud de la respuesta final, con la contraseña password, es claramente diferente, por lo que nos podría dar una pista de que esta contraseña es la correcta.

Aquí en ésta petición correcta, la Response 1 nos redirige al index.php, en los casos anteriores nos redirige a login.php, por lo que podemos concluir que esta es la contraseña correcta.

Posteriormente en la Request 2 vemos que nos hace la redirección a la sección autenticada de la aplicación y nos ha configurado las cookies de sesión.

Password spraying

Execution of several attempts of authentication with one known password and different users

Previous information gathering

- **Default passwords**
- **Patterns**

Tools

- **BurpSuite -> Intruder**

Mitigation

- **Blocking IP address when it is received several requests**

Otro tipo de ataque es un poco a la inversa, se trata del llamado password spraying, en lo que consiste esta prueba es en la ejecución de muchos intentos de autenticación pero conociendo la contraseña, no el usuario.

De forma que si tenemos un conocimiento de que la empresa o la plataforma en la aplicación pueda estar usando un tipo de contraseñas por defecto o con patrones, vamos a tener una inteligencia que llamamos ya sobre hecha para realizar este tipo de pruebas.

Entonces recordamos que fuerza bruta se trata de muchos intentos de autenticación de un usuario con muchas contraseñas.

En el caso de Password spraying se trata de intentos de autenticación de una contraseña con muchos usuarios.

Para ello también tenemos que tener en cuenta que se debe tener una recolección de información relativa a posibles usuarios que haya autenticados en la aplicación.

Es decir, si sabemos que hay un patrón de contraseñas o el uso de contraseñas débiles en una aplicación, lo que tenemos que hacer también es intentar identificar

no solamente usuarios que puedan encontrarse existentes en la aplicación, sino también posibles usuarios cuyo nombre pueda ser adivinable.

En este caso también la herramienta usada más conocida es burpsuite con su funcionalidad Intruder.

En este caso la mitigación que se recomienda es el bloqueo no del usuario sino de la dirección IP desde la que son enviadas las peticiones, ya que no podemos bloquear a un usuario porque cada petición se realiza con un usuario diferente y con una misma contraseña. Aquí lo que bloqueamos no es el usuario sino la dirección IP desde la que se realizan todas las peticiones al servidor.

Práctica:

También para realizar un ejemplo de esta prueba vamos a hacer uso de la aplicación de DVWA como entorno víctima y de nuestra máquina Kali como máquina atacante.

En este caso, vamos a hacer uso por ejemplo del usuario root, pero sabemos que la contraseña es password. Nos muestra el mensaje de login fallido, pero tal y como pasaba en el caso anterior, tenemos la petición de envío de las credenciales identificada en Burp suite y lo que hacemos es enviarla al intruder.

En este caso el valor que va a cambiar es el nombre del usuario, por lo que es lo que seleccionamos con Add.

Y en este caso, en la tab Payload, lo que vamos a probar es root, administrator, administrador, test, guest, estos nombres de usuario que suelen estar configurados por defecto en muchos sistemas. Y por último vamos a probar con admin.

En este caso, en la tab Settings, vamos a configurar también la redirección de las peticiones que son enviadas y de sus respuestas y vamos a comenzar nuestro ataque.

4. Intruder attack of http://192.168.186.132

Results Positions Payloads Resource pool Settings

Filter: Showing all items

Request ^	Payload	Status code	Response received	Error	Redirects followed	Timeout	Length
0		200	154		1		1675
1	root	200	75		1		1675
2	administrator	200	79		1		1675
3	administrador	200	134		1		1675
4	test	200	161		1		1675
5	guest	200	186		1		1675
6	admin	200	67		1		4932

Como podemos ver ocurre como en el caso de la fuerza bruta, la longitud de la respuesta. La diferencia de esta longitud es significativo, por lo que en este caso vemos que usando el usuario guest nos sigue redirigiendo a login.php con administrator ocurre el mismo caso, pero con el usuario admin, aquí nos redirige ya a index.php de forma que nos vuelve a redirigir a la sección autenticada de la aplicación y estaríamos satisfactoriamente autenticados.

Password policy

Requirement where a password must accomplish rules.

Designed to improve the internal security.

Examples of rules for password policies:

- Minimum length
- Complexity
- (Periodically) expiration dates
- Reuse

Tool

- <https://www.passwordmonster.com/>

Tras esto lo que vamos a tratar ahora es la política de contraseñas, que es un aspecto del que es necesario hablar, ya que se trata de los requisitos que se deben cumplir en una contraseña para que la contraseña sea aceptada para su uso en una aplicación, de forma que esta medida de seguridad lo que busca al final es evitar el

uso de credenciales débiles o que sean fácilmente adivinables, de modo que las cuentas de los usuarios tengan otra capa más de seguridad.

También hay que tener en cuenta que el nivel de complejidad de las contraseñas en algunas ocasiones viene de la mano del nivel de sensibilidad de la información que se gestione en la aplicación concretamente, pero también depende de la infraestructura que se encuentre gestionando por detrás esas aplicaciones.

Entre las políticas de contraseñas podemos encontrar la longitud, que en muchos casos nos viene dada por un mínimo de 4 u 8 caracteres, de 12 o 16 según el tipo de aplicación, como indicamos hace un momento.

En relación con la complejidad se refiere al uso de minúsculas, mayúsculas, números, caracteres especiales, pueden ser los cuatro casos, pueden ser tres según nos indique la aplicación.

En relación con la expiración de la contraseña, es un caso que suele darse principalmente en entornos corporativos para cambiar la contraseña con cierta periodicidad.

Y también en relación con esta expiración de la contraseña viene el cuarto punto que es el reuso de la contraseña, ya que al solicitar que se cambien las contraseñas una condición obligatoria es que la contraseña nueva no sea la misma que la contraseña que se encontraba anteriormente o que no coincida la contraseña nueva con las cuatro o seis últimas contraseñas que se habían usado en la plataforma.

Para ello hay una aplicación, una plataforma llamada passwordmonster que nos ayuda, nos invita a que introduzcamos contraseñas que pensamos usar y nos dice el nivel de complejidad que tienen estas contraseñas.

Para ello vamos a verla también passwordmonster.com y en este caso, pues vamos a probar por ejemplo la contraseña 1234 en la que nos dice ya que es muy débil, podemos ampliarla a 12345678. Nos indica que estas contraseñas son muy muy fácilmente adivinables y no se tardaría nada en crackearlas.

Por ejemplo podríamos poner administrator, ya nos está diciendo que con caracteres, con letras se tarda ligeramente más en crackear las contraseñas que simplemente con números.

Si además ampliamos un poquito más la complejidad de esta contraseña, administrator123 va añadiendo caracteres, vamos añadiendo complejidad a esta contraseña vamos a tardar más en crackearla a la hora de usarla como contraseña en una plataforma.

Si además en vez de usar solamente caracteres en minúsculas, letras en minúsculas, añadimos mayúsculas, vamos a ampliar también la complejidad.

Si en el caso por ejemplo podemos cambiar la i por un 1, aquí ya ampliamos, empezamos a meter un poquito de entropía, de aleatoriedad en las contraseñas y cuanto más aleatorias sean las contraseñas, más difícil y más complicado va a ser poder obtener esas contraseñas.

Como conclusiones de esta sesión podemos indicar los siguientes puntos.

En primer lugar podemos decir que los intentos de autenticación, los intentos de login en un formulario de autenticación en una aplicación web deben estar limitados entre 3 5 intentos de login, de forma que si un usuario intenta identificarse en una aplicación realizando varios intentos, pues este usuario se vería afectado con un bloqueo temporal, sin poder autenticarse la aplicación hasta que pase este tiempo de bloqueo.

Por otra parte hay que tener en cuenta que para evitar los ataques de password spreading lo podemos mitigar evitando exponer o limitando la visibilidad de los usuarios que se encuentren registrados en la aplicación sin que el usuario se haya autenticado.

Con esto quiero decir, yo no puedo ver el resto de usuarios que puedan existir en una aplicación a menos que yo me autentique previamente como usuario existente también en la aplicación.

De esta forma no vamos a dar visibilidad a usuarios que ya estén existentes y las credenciales de estos usuarios sean más o menos robustas, se verían reducidos estos intentos de ataques de fuerza bruta sobre estos usuarios.

Por otra parte, también podemos mitigar ataques de fuerza bruta ya sea con el bloqueo temporal que hemos dicho anteriormente del usuario con con el que se están realizando multitud de intentos de autenticación, como también con el bloqueo de la dirección IP con la que se están realizando la multitud de peticiones con la dirección con la que se están enviando las peticiones al servidor web y que se están replicando en estos intentos de adivinar las credenciales válidas.

Por último punto, es altamente recomendable y es muy importante hacer un inciso en que es importante implementar políticas de contraseñas en las aplicaciones.

¿Por qué? Porque a nosotros como auditores, en el caso de no haber políticas de contraseñas, nos facilita realizar pruebas con credenciales débiles.

¿Esto que implica a los usuarios y a los servidores de estas aplicaciones? Que la confidencialidad de los usuarios se ve afectada porque podemos suplantar la entidad de estos usuarios que hagan uso de credenciales débiles porque la aplicación no obligue a usar una política de contraseña robusta.

Entonces obligando a hacer uso de esta política de contraseñas lo que se consigue es que las contraseñas no sean débiles ni por defecto por parte de los usuarios y sea mucho más complejo obtener credenciales válidas en estas plataformas.

Autorización: Roles y directory traversal

En esta sesión vamos a tratar una de las principales vulnerabilidades que se dan en la autorización y en el acceso indebido a recursos en servidores, principalmente en servidores web.

Para ello vamos a hablar sobre lo que son los roles y la importancia que tienen y la vulnerabilidad de Directory Traversal.

Roles

Different user roles must have different access to internal resources.

Access Control List (ACL) identifies which users are supposed to be able to have access to resources.

Principle of least privilege (PoLP)

- A user is given the minimum levels (or permissions) of access necessary to perform his or her job functions.

Para ponernos en contexto para esta sesión debemos saber y que nos quede claro lo que son los roles y el papel que juegan, que lo que hacen es hacer referencia al tipo de trabajo y las tareas que realiza un usuario. De forma que si lo llevamos al contexto de una aplicación o al ámbito corporativo, incluso las personas de diferentes departamentos deben tener limitado el acceso a diferentes recursos, ya se trate a nivel digital como puedan ser carpetas, ficheros o servidores o redes o salas de reuniones, también deben tenerlo en estos casos.

Lo mismo ocurre con los accesos de los usuarios que no se encuentran autenticados en las aplicaciones, que deben ser diferentes.

Este acceso de recursos a aquellos usuarios que sí se encuentren autenticados y a su vez que sean también estos accesos diferentes a aquellos usuarios que aún están autenticados también tengan privilegios.

Entonces, para implementar esta diferenciación de roles y poder llevarla a cabo, existen lo que se conocen como listas de control de acceso (ACL), que sirven para identificar por una parte y limitar por otra aquellos usuarios que deben tener acceso a qué recursos, de forma que según el nivel de sensibilidad de los recursos, las listas de control correspondientes serán más o menos permisivas.

Con esta técnica de las listas de control de acceso se aplica lo se conoce como el principio de menor privilegios, que consiste en aplicar o en asignar el menor nivel de privilegios posible a un usuario o a un recurso, o en este caso asignar la menor cantidad de privilegios posible a un usuario para que pueda hacer una determinada tarea.

Directory Traversal

Aims to access to restricted files and directories, even execute commands

Could provide an attacker more information required to further compromise the system

The commands executed would impersonate the attacker as the user which is associated with "the website".

- Depends on the privileges of the user in the system

En este punto también vamos a hablar de la vulnerabilidad de Directory Traversal, que se produce por una gestión incorrecta de accesos a recursos internos de un servidor.

En el caso de esta vulnerabilidad también lo que se pretende es obtener acceso a recursos pero que están más allá de la estructura de ficheros de la aplicación web, es decir, lo que hace es acceder no solamente a ficheros que puedan estar dentro de la configuración de la aplicación web o del servidor web, sino que también puede acceder a directorios y ficheros del sistema operativo del servidor.

Entonces además al tener acceso a estos ficheros del sistema operativo, también lo que pasa es que es posible realizar ejecución de comandos, ya que podemos

conocer las rutas en las que se encuentra la consola de Windows, en el caso de sistemas operativos Windows, pues se encuentra el fichero C:\windows\system\cmd.exe. Y el caso paralelo puede encontrarse en sistemas operativos de Unix que se puede encontrar el fichero /opt/bin/bash y podríamos concatenar la ejecución de estos ficheros con el comando que deseemos ejecutar.

Otro aspecto que tenemos que saber es que a la hora de ejecutar los comandos, el nivel de privilegios del usuario con el que se ejecutan dichos comandos es el usuario asociado a la aplicación y en muchos casos se trata del usuario ww-data, por ejemplo en el caso de Linux.

Ejemplos:

Algunos de los ejemplos de explotación de esta vulnerabilidad los vamos a ver a continuación, donde podemos ver diferentes vías de acceso a recursos internos. Vamos a verlos uno a uno.

[Website.tld/resource?view=../../../../Windows/system.in](#)

En éste primer caso vemos que vamos a retroceder no la carpeta raíz, sino diferentes carpetas. Una vez llegamos al directorio de Windows, vamos a acceder al fichero [system.in](#).

[Website.tld/resource?view=%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd](#)

En éste segundo ejemplo lo que vamos a ver es la opción de la codificación en URL para retroceder al fichero al directorio raíz. En la codificación URL [%2e](#) simboliza al punto, entonces al poner [%2e%2e/](#) lo que estamos indicando es [../](#) que es lo que va a interpretar el navegador, que lo va a traducir y lo va a procesar el servidor. De esta forma entonces vamos a retroceder hasta la carpeta raíz, de ahí a etc y se va a acceder al fichero passwd.

[Website.tld/resource?view=backup.db](#)

En éste tercer caso lo que se realiza directamente es la solicitud a un archivo determinado del sistema. En este caso hemos puesto el ejemplo en el que

asumimos que el fichero podemos ver que es una copia de seguridad de una base de datos, también para ver el impacto que puede llegar a tener esta vulnerabilidad con los datos sensibles que se pudieran almacenar en este caso.

```
Website.tld/scripts/..%5c../opt/bin/bash+pwd
```

En éste cuarto ejemplo lo que hacemos es codificar la contrabarra con %5c para poder otra vez retroceder al fichero raíz opt/bin/bash y posteriormente ejecutar el fichero pwd.

```
Website.tld? filename=../../../../etc/passwd%00.png
```

En éste último caso lo que se intenta es acceder al fichero passwd pero haciendo uso del carácter nullbyte y una extensión de un fichero cualquiera, en este caso png, y en este caso lo que pasa es que el navegador va a interpretar hasta el nullbyte, de forma que se va a tener acceso otra vez a este fichero passwd.

Mitigaciones a la vulnerabilidad de Directory Traversal

- Parametrize input data when it's sent to the server
- Implement ACLs restricting access to remote files

Como mitigaciones también de esta vulnerabilidad podemos anotar que es necesario en primer lugar parametrizar las entradas de texto antes de que sean procesadas por el servidor, de forma que los valores que son enviados al servidor sean interpretados como cadenas de texto y no como comandos u órdenes.

De la misma forma, otra mitigación es implementar las listas de control de acceso que mencionábamos al principio, restringiendo el acceso a recursos, a los roles o a los usuarios concretos necesarios y de esta forma cualquier usuario no podría tener acceso a ficheros de sistema operativo en el caso de estos ejemplos.

Como conclusiones de esta sesión podemos decir que es necesario identificar, asociar y diferenciar correctamente los usuarios y los roles que puedan encontrarse no solamente en un servidor de una aplicación, sino también de una infraestructura.

Ver aquellos que necesitan más privilegios, los que necesitan menos privilegios y según sus roles o el tipo de usuarios que sean, definir listas de control de acceso para distribuir los recursos internos según sea necesario y evitar accesos que no sean permitidos a información que no corresponda por parte del usuario.

De la misma forma y de cara al directory traversal, lo que debemos recordar que lo fundamental es parametrizar la información que nos llega de elementos de entrada de texto y parametrizarla antes de que sea procesada por el servidor.

Errores de autenticación: IDOR y escalada de privilegios

En el ámbito de la ciberseguridad, vamos a explorar las vulnerabilidades relacionadas con la autorización en aplicaciones web y servidores, enfocándonos en dos problemas específicos: **Insecure Direct Object References (IDOR)** y la **escalada de privilegios**.

Insecure Direct Object References (IDOR)

La vulnerabilidad

IDOR se produce cuando una aplicación permite a los usuarios acceder directamente a objetos o recursos que no deberían poder ver o manipular. Esto ocurre porque los recursos tienen identificadores predecibles o fáciles de adivinar, como números consecutivos o nombres por defecto. Los recursos afectados no tienen restricciones de acceso o visibilidad, lo que permite a un usuario no autorizado acceder a archivos o información que pertenece a otros usuarios, ya sea con el mismo nivel de privilegios o con uno diferente.

Para identificar y explotar esta vulnerabilidad, se usan ataques de

fuerza bruta o **fuzzing**, enfocándose en los identificadores de los recursos. La herramienta más utilizada para esto es

Burp Suite Intruder, que permite usar diccionarios o rangos de identificadores numéricos para automatizar la búsqueda de recursos vulnerables.

Ejemplos de Explotación IDOR

Aquí se presentan algunos ejemplos comunes de cómo se manifiesta esta vulnerabilidad en las URL de una aplicación web:

- **Website.tld?author=1** En este caso, el parámetro **author** utiliza un identificador numérico consecutivo (**1**) que puede ser fácilmente adivinado. Al modificar este valor, un atacante podría intentar acceder a la información de otros autores.
- **Website.tld?user=guest** Este ejemplo muestra un identificador basado en el nombre (**guest**). Al cambiar este valor por otros nombres de usuario por defecto o adivinables, un atacante podría intentar identificar la existencia de otras cuentas.
- **Website.tld?invoice=123456** Aquí, se usa un número de factura predecible. Si un atacante cambia el valor **123456** por **123457** o **123458**, podría acceder a las facturas de otros usuarios.
- **Website.tld?signedDocument=872631** En este caso, un documento confidencial tiene un identificador numérico de seis dígitos. Un atacante podría usar la función Intruder de Burp Suite para probar todas las combinaciones posibles y acceder a documentos sensibles que no le corresponden.

Mitigación de IDOR

Las principales recomendaciones para mitigar los ataques IDOR son:

- **Bloquear las direcciones IP de origen:** Dado que estos ataques implican una gran cantidad de peticiones, bloquear las direcciones IP desde las que se originan puede detener la prueba.
 - **Establecer controles de acceso:** Es crucial establecer reglas de acceso rigurosas para los recursos, asegurándose de que solo los usuarios a los que les corresponden puedan verlos. De esta forma, se evita que otros usuarios no autorizados accedan a la información sensible.
-

Escalada de privilegios

La **escalada de privilegios** es un concepto amplio en ciberseguridad que busca obtener acceso no autorizado a un sistema. Se logra cuando un atacante realiza acciones o accede a recursos que están restringidos para su nivel de usuario actual. Esto podría incluir la ejecución de comandos o el acceso a archivos y directorios a los que no debería tener permiso. Lograr esto requiere un conocimiento profundo del objetivo y su infraestructura interna.

Para llevar a cabo una **escalada de privilegios**, es crucial tener un conocimiento profundo del sistema o aplicación vulnerable, así como de su infraestructura interna. Aunque no es estrictamente necesario, el conocimiento a bajo nivel de los sistemas operativos es muy común y beneficioso para explotar estas vulnerabilidades.

Vectores de Ataque Comunes

Existen varias vías para lograr una escalada de privilegios:

- **Compromiso de Cuentas:** Si las credenciales de un usuario son débiles, se pueden adivinar o robar, lo que permite al atacante suplantar la identidad de la víctima y obtener sus privilegios en el sistema. Esto se ve facilitado cuando no hay políticas de contraseñas robustas o un segundo factor de autenticación.
- **Vulnerabilidades en el Software:** Los fallos de configuración en las aplicaciones de software o el uso de versiones obsoletas con vulnerabilidades conocidas son un vector de ataque importante. Estas fallas pueden ser explotadas para elevar los privilegios del atacante en el sistema.
- **Software Malicioso (*Malware*):** Los programas maliciosos pueden ser descargados, instalados o enviados a equipos para infectarlos y realizar tareas sin el consentimiento del usuario legítimo.
- **Ingeniería Social:** Esta técnica se basa en persuadir a una persona para que realice una acción que no debería. A menudo se utiliza para engañar a los

usuarios y que revelen información sensible o que instalen *malware*. Los métodos de ingeniería social incluyen:

- **Phishing:** A través de correos electrónicos.
 - **Smishing:** A través de mensajes de texto (SMS).
 - **Llamadas telefónicas:** Haciéndose pasar por alguien de confianza (por ejemplo, un jefe o una entidad financiera).
-

Conclusiones Clave

- Para mitigar vulnerabilidades como la **Insecure Direct Object References (IDOR)**, la mejor práctica es asegurarse de que los identificadores de los recursos no sean predecibles ni fáciles de adivinar.
- La escalada de privilegios puede tener una gran variedad de orígenes. Puede ser causada por fallos en las tecnologías utilizadas por una aplicación, la configuración de los recursos, o los permisos de los usuarios en los servidores que las gestionan.